

MAREVLO · PRODUCTION RAG

Debugging RAG

The single most valuable skill in the field — telling exactly which step failed, so you fix the real problem instead of guessing.

PART II — MAKING IT WORK

Module 6

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

In Module 5 we built a working RAG system, and we ended on a hard question: the moment it gives a wrong answer, how do we even know *why*? Was the right information never retrieved, or was it retrieved and then mishandled? This module answers that question, and the skill it teaches is, without exaggeration, the most valuable one in all of RAG work. It is the difference between someone who hopes their system works and someone who knows why it does — and who can fix it fast when it does not.

Here is why this skill matters so much. A wrong RAG answer almost always looks the same from the outside: the user just sees a bad answer. But the *cause* could be in any of several different stages, and the fix is completely different depending on which. Without a method for telling them apart, you end up changing things at random — tweaking the prompt, swapping the model, adjusting settings — and hoping something helps. That random guessing is the single most expensive habit in the field, and this module replaces it with a calm, ordered procedure that finds the real cause in minutes.

THE WHOLE MODULE IN ONE SENTENCE

When an answer is wrong, do not guess and do not touch the prompt yet — first *print the retrieved chunks*, because that one look tells you whether the problem is in retrieval (the information was never found) or in generation (the information was found and mishandled), and those two have completely different fixes.

Where we are

This module is a skill, not a feature — it does not change what the system does, it changes how you find and fix what is wrong with it. It pairs naturally with observability (Module 8): observability is about capturing the data of each request, while debugging is the method of *using* that data to diagnose a fault. You can practise everything here with simple printing; once you have traces from Module 8, the same method just gets faster.

SECTION 1

The cardinal rule: separate retrieval from generation

Almost every RAG bug is solved by one habit — looking at what was retrieved before touching anything else.

Recall the journey of a question through our system: it is embedded, the store is searched for the nearest chunks, those chunks are assembled into a prompt, and the model generates an answer. Now group those four steps into two halves. The first half, **retrieval**, is everything up to and including finding the chunks. The second half, **generation**, is everything from assembling the prompt to producing the answer. Almost every wrong answer is a failure in one half or the other — and the cardinal rule of debugging is to find out *which half* before doing anything else.

The way you find out is beautifully simple: **print the chunks that were retrieved for the failing question**. That single action splits the problem cleanly in two. If the chunk that should have answered the question is *not* among those retrieved, then the model never had the information — this is a retrieval failure, and no change to the prompt or the model can possibly fix it. If the right chunk *is* among those retrieved but the answer is still wrong, then the information was there and something downstream mishandled it — this is a generation failure, and now the prompt and model are fair game. One look, and you know which world you are in.

The one fork that resolves most RAG bugs

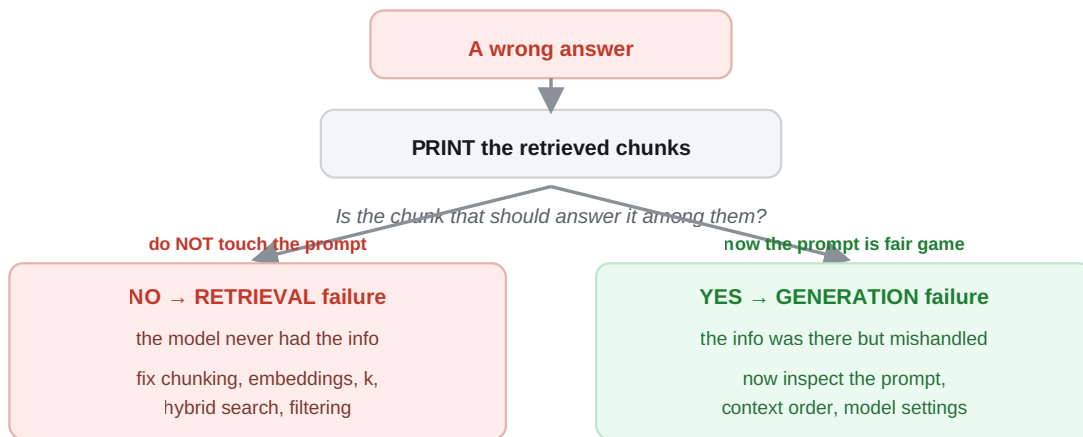


Figure 1. The single fork that resolves the majority of RAG bugs. Printing the retrieved chunks tells you which half failed — and the two halves have completely different, non-overlapping fixes.

ANALOGY — THE COOK AND THE PANTRY

A dish comes out wrong. Before blaming the cook's technique, you check the pantry: were the right ingredients even there? If the recipe needed lemons and there were no lemons, no amount of skill at the stove could have saved the dish — the failure was in the pantry, not the cooking. Only once you confirm the ingredients were present does it make sense to examine how they were cooked. Printing the retrieved chunks is checking the pantry. Tweaking the prompt before you check it is criticising the cook for a missing lemon.

LAB 6.1 — The first thing you do for any wrong answer (our own)

```
def inspect_retrieval(question):
    """Print exactly what the retriever found for a question.

    This is the single most useful debugging action in RAG. Run it FIRST,
    before changing anything, for every wrong answer.
    """
    results = store.similarity_search_with_relevance_scores(question, k=8)

    print(f"Question: {question}\n")
    print("Retrieved chunks (best first):")
    for document, score in results:
        print(f"  [{score:.2f}] {document.page_content[:90]}")

    # Now ask yourself the only question that matters at this point:
    print("\n--> Is the chunk that should answer this in the list above?")
    print("    NO = retrieval problem (do not touch the prompt)")
    print("    YES = generation problem (now inspect the prompt)")

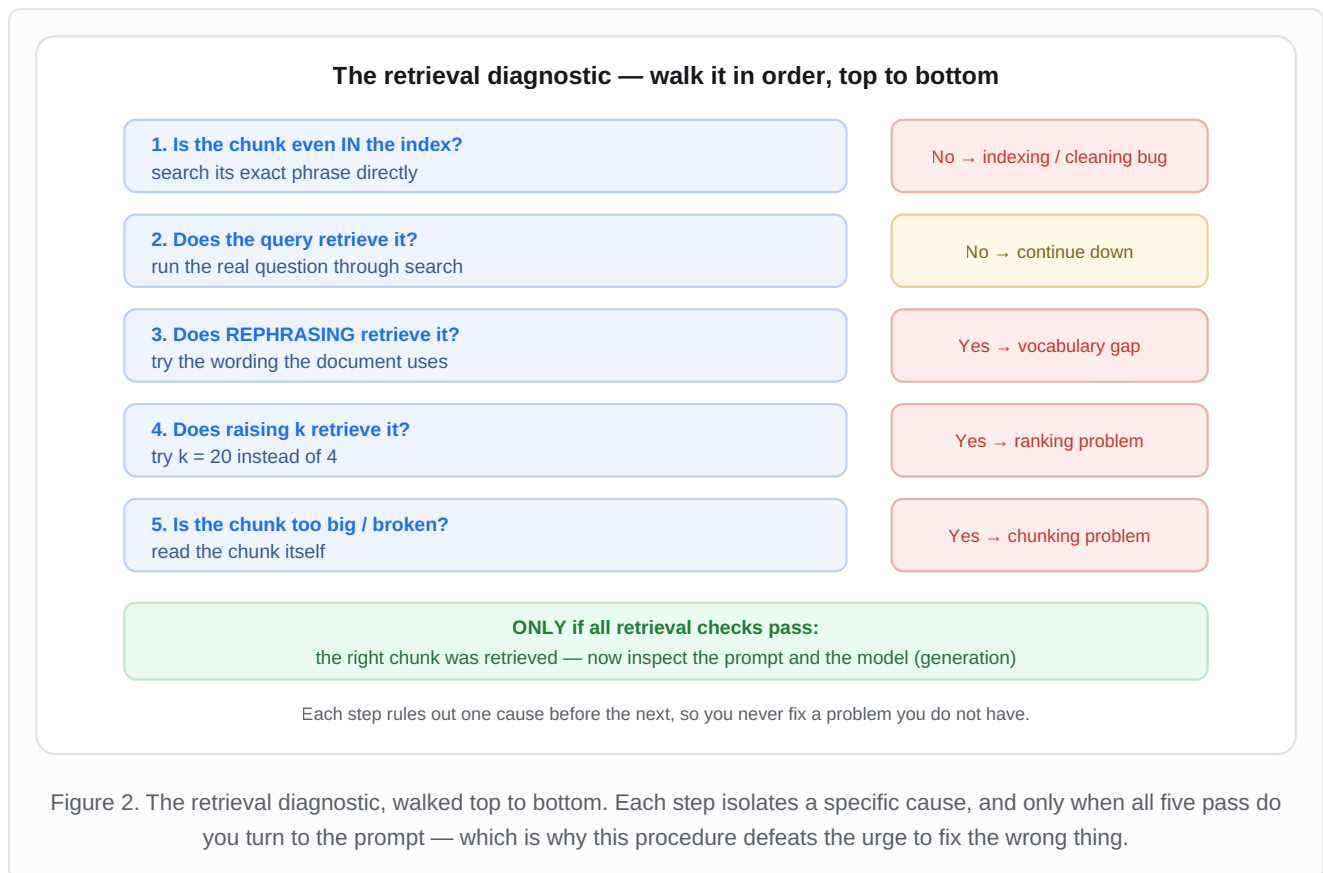
inspect_retrieval("What does the warranty cover?")
```

SECTION 2

The diagnostic procedure, step by step

Once you know it is a retrieval problem, a short ordered checklist pinpoints exactly which retrieval problem — without guessing.

The fork in Section 1 tells you which half failed. If it is a generation failure, you move on to inspecting the prompt, which we will come to. But if it is a retrieval failure — the more common case — there is a short, ordered procedure that narrows down exactly *which* retrieval problem you have. The order matters: each step rules out a cause before moving to the next, so you never waste effort on a fix for a problem you do not have. Walk it from the top every time.



Let us walk the five steps in words, because understanding *why* each one isolates what it does is what lets you apply them confidently. **Step one** asks whether the chunk is even in the index at all — you search for an exact phrase you know is in it. If it does not come back, the problem is upstream in indexing or cleaning: perhaps the document was never loaded, or the chunk was stripped as junk, or the index is stale. No retrieval tuning helps if the chunk is simply not there. **Step two** runs the real question and confirms the failure. **Step three** rephrases the question using the words the document actually uses; if that suddenly retrieves the chunk, you have a vocabulary gap between how users ask and how documents are written — the cure is query transformation or hybrid search. **Step four** raises the number of chunks retrieved; if the right chunk appears at, say, position seven, it was findable all along but ranked too low — a ranking problem, cured by re-ranking. **Step five** reads the chunk itself; if it is oversized, broken, or stripped of context, the cure is back in chunking. And only when all five pass — the right chunk is indexed, retrieved, well-ranked, and whole — do you finally turn to the prompt.

LAB 6.2 — The diagnostic as a runnable checklist (our own)

```
def diagnose_retrieval(question, known_phrase):
    """Walk the retrieval diagnostic for one failing question.

    `known_phrase` is a distinctive phrase from the chunk you believe
    SHOULD answer the question -- it is how we check each step.
    """
    # Step 1: is the chunk even in the index? Search its exact phrase.
    in_index = store.similarity_search(known_phrase, k=3)
    if not any(known_phrase.lower() in d.page_content.lower() for d in in_index):
        return "Step 1 FAILED: chunk not in index -> indexing/cleaning bug."

    # Step 2: does the real question retrieve it at the normal k?
    normal = store.similarity_search(question, k=4)
    if any(known_phrase.lower() in d.page_content.lower() for d in normal):
        return "Retrieval is fine -> this is a GENERATION problem. Inspect the prompt."

    # Step 4: does raising k surface it? (ranking problem if so)
    wide = store.similarity_search(question, k=20)
    if any(known_phrase.lower() in d.page_content.lower() for d in wide):
        return "Step 4: found at high k -> RANKING problem. Add re-ranking."

    # If still missing, it is a vocabulary or chunking issue (steps 3 & 5).
    return "Not retrieved even at k=20 -> vocabulary gap or chunking. Try rephrasing; read the chunk."

print(diagnose_retrieval("What does the warranty cover?", "two-year limited warranty"))
```

SECTION 3

When it is a generation problem

If the right chunk was retrieved and the answer is still wrong, the fault is downstream. Here is how to narrow it.

Suppose the diagnostic clears retrieval: the right chunk was indexed, retrieved, well-ranked, and whole, yet the answer is still wrong. Now — and only now — the problem is in generation, and you have earned the right to look at the prompt and the model. There are a few common generation faults, and the way to tell them apart is to look at the assembled prompt and the answer together.

If the answer contradicts the retrieved chunk and instead matches general knowledge, the model is ignoring your context — the cure is a firmer grounding instruction. If the answer invents detail that is in no chunk, the model lacked permission to say "I don't know." If the answer is right but inconsistent between runs, the temperature is too high. If the model seems to be following an instruction that came from *inside* a document, your prompt is not separating instructions from data clearly enough. In every case, the move is the same: look at the exact prompt that was sent and the exact answer that came back, side by side, and the mismatch will point to the fault. This is why recording the assembled prompt — part of a good trace from Module 8 — is so valuable.

THE HABIT THAT MAKES GENERATION BUGS VISIBLE

During development, print the full assembled prompt before sending it, and read it as if you were the model. Most generation bugs are obvious the moment you actually look at what the model received: the context is missing, or the instructions are buried, or a document is giving orders. The bug hides only as long as you refuse to look at the prompt.

SECTION 4

Anatomy of three debugging sessions

Watch the method in action. Each session starts with the same symptom — a wrong answer — and the procedure leads to a different real cause.

The procedure only truly clicks when you see it trace a wrong answer back to its real cause. Below are three sessions. Notice that all three begin identically — a wrong answer and the temptation to blame the model — and that in every case the first real step is the same, yet the procedure leads to three completely different fixes, in three different parts of the system.

Session A — "It keeps saying it doesn't know about the warranty"

- 1 **Temptation:** "the model is being unhelpful — let me soften the prompt." Resist it. Print the chunks first.
- 2 **Step 1 of the diagnostic:** search the exact phrase "two-year limited warranty." It does not come back — the chunk is not in the index at all.
- 3 **Cause found:** the warranty page was a scanned image, and the loader extracted nothing usable from it. This is an indexing failure, the very first step.
- 4 **Fix:** re-load the warranty page with proper text extraction. **No prompt change was ever needed** — softening the prompt would have done nothing, because the information was never in the system.

Session B — "It answers warranty questions with cleaning instructions"

- 1 **Print the chunks:** the warranty chunk *is* in the retrieved set — but at position six, while four cleaning chunks rank above it.
- 2 **Step 4 of the diagnostic:** raise *k* to 20; the warranty chunk is clearly there, just ranked too low to make the top four.
- 3 **Cause found:** a ranking problem — the fast search ranked a genuinely relevant chunk just outside the cut-off.
- 4 **Fix:** add re-ranking (Module 9), which promotes the truly relevant chunk to the top. Again, the prompt was never the problem.

Session C — "It says the warranty is three years, but our docs say two"

- 1 **Print the chunks:** the correct "two-year" chunk is right there at the top, retrieved perfectly. Retrieval is clean.
- 2 **Now** — and only now — inspect the prompt and answer together. The chunk says two years; the answer says three.
- 3 **Cause found:** the prompt had no firm grounding instruction, so the model overrode the context with its general knowledge that "most warranties are three years." A genuine generation failure.
- 4 **Fix:** add "answer using ONLY the context" to the prompt (Module 5). This time the prompt *was* the problem — but you only know that because retrieval was cleared first.

Three identical symptoms, three different causes, three different fixes — and in two of the three, touching the prompt first would have wasted hours. That is the entire case for the method: the symptom never tells you the cause, but the procedure always does.

FAULTS AND CORRECTIONS

The debugging habits that waste the most time

For each: the broken habit, why it costs you, and the fix.

FAULT 1 — Tweaking the prompt before checking what was retrieved

THE BROKEN HABIT

```
# A wrong answer appears, so the first move is to reword the prompt --  
# again and again -- without ever looking at the retrieved chunks.  
for wording in prompt_variations:  
    test(wording) # but the right chunk was never retrieved
```

WHY IT COSTS YOU

This is the most common and most expensive debugging mistake in RAG. If the right chunk was never retrieved, the prompt cannot use information it never received, so every rewording fails for the same hidden reason — and you can lose days this way. The prompt is the *last* thing to suspect, not the first.

THE CORRECTION

```
# Always print the retrieved chunks FIRST. Only consider the prompt once  
# you have confirmed the right chunk was actually retrieved.  
inspect_retrieval(question) # is the right chunk even there?
```

FAULT 2 — Trusting the answer's confidence as a sign it is right

THE BROKEN HABIT

```
# "The answer sounds detailed and confident, so retrieval must be fine."  
# Confidence is judged by tone, not by checking the chunks.  
if answer_sounds_confident(answer):  
    assume_retrieval_is_fine() # a dangerous assumption
```

WHY IT COSTS YOU

A model writes a fabricated answer in exactly the same confident tone as a correct one — fluency is not accuracy. Judging correctness by how assured the answer sounds will route you to the wrong half of the system, because the most dangerous wrong answers are precisely the confident ones built on no real context. Tone tells you nothing; the retrieved chunks tell you everything.

THE CORRECTION

```
# Verify against the chunks, never against the tone. Check the answer's  
# claims against what was actually retrieved.  
results = store.similarity_search_with_relevance_scores(question, k=8)  
# Does a retrieved chunk actually support the answer's claim? If not, dig in.
```

FAULT 3 — Changing several things at once while debugging

THE BROKEN HABIT

```
# In one go: adjust chunk size, raise k, AND reword the prompt. The bug  
# goes away -- but which change fixed it? Unknown.  
adjust_chunking(); raise_k(); rewrite_prompt()  
test(question)  # "fixed" but you learned nothing
```

WHY IT COSTS YOU

If you change three things and the bug disappears, you cannot tell which change mattered, so you cannot apply the lesson elsewhere, and you may have introduced two new problems masked by one fix. Worse, you may keep an unnecessary, costly change forever because it happened to be bundled with the real fix. Debugging, like optimisation, only teaches you something when you change one thing at a time.

THE CORRECTION

```
# Change exactly one thing, test, and confirm THAT was the cause before  
# moving on. One variable at a time keeps cause and effect clear.  
raise_k(); test(question)  # did this alone fix it? now you know.
```

FAULT 4 — Never checking whether the chunk is even in the index

THE BROKEN HABIT

```
# Debugging the retriever's ranking and settings for hours -- when the  
# chunk was never indexed in the first place (bad load, stale index, junk-stripped).  
tune_retriever_settings()  # tuning how to find something that isn't there
```

WHY IT COSTS YOU

If the chunk is not in the index at all, no amount of retriever tuning, re-ranking, or query rewriting will ever surface it — you are tuning how to find a thing that does not exist. This is why Step 1 of the diagnostic is to search the exact phrase: it instantly rules out the whole category of "it was never there," which would otherwise waste every downstream effort.

THE CORRECTION

```
# Step 1, always: confirm the chunk exists in the index by its exact phrase.  
hits = store.similarity_search("two-year limited warranty", k=3)  
# If it is not here, the bug is in indexing/cleaning -- fix that, not the retriever.
```

FAULT 5 — Debugging from the report instead of reproducing the failure

THE BROKEN HABIT

```
# A user reports a vague "the warranty answer was wrong." We start  
# changing things based on the description, without reproducing it.  
guess_from_description("warranty answer was wrong")
```

WHY IT COSTS YOU

You cannot fix what you cannot see happen. A second-hand description omits the exact question, the retrieved chunks, and the prompt — the very things you need. Changing the system based on a vague report is guessing twice over. Reproduce the exact failing question first, so you have real chunks and a real answer to examine, not a memory of one.

THE CORRECTION

```
# Reproduce the exact failing question, then run the diagnostic on it.  
# A trace (Module 8) is ideal: it captures the real chunks and prompt.  
inspect_retrieval(the_exact_question_the_user_asked)
```

Checkpoint: try it yourself

This exercise builds the reflex at the heart of the module — checking retrieval before touching anything else — by having you deliberately create each kind of failure and diagnose it. It takes about thirty-five minutes and is the most practical debugging practice in the course.

1. Create a retrieval failure on purpose: remove one chunk from your store, then ask a question that depended on it. Run `inspect_retrieval` and confirm — by looking, not guessing — that the right chunk is simply absent. That is Step 1 catching an indexing failure.
2. Create a ranking failure: add the chunk back, then ask a question worded quite differently from the chunk. If it does not appear in the top four, raise `k` to 20 and watch it appear lower down. That is Step 4 catching a ranking problem.
3. Create a generation failure: with the right chunk retrieved cleanly, remove the "use only the context" instruction from your prompt and ask something your model has strong general knowledge about. Watch it override the correct chunk. That is the fork landing on the generation side.
4. For each of the three, write down the one-line diagnosis and the correct fix. Notice that all three started as "a wrong answer," and only the procedure told them apart.

WHAT YOU SHOULD NOTICE

Three times you started with the same symptom, and three times the first action was identical — look at what was retrieved. That single reflex, applied before any change, is what separates calm, fast debugging from hours of frustrated guessing. Build that reflex and you have built the most valuable skill in RAG.

What to carry forward

THE SIX THINGS TO REMEMBER

One, when an answer is wrong, print the retrieved chunks before doing anything else — that one look splits the problem into retrieval versus generation, which have completely different fixes. **Two**, if the right chunk was not retrieved, it is a retrieval problem and the prompt cannot help — do not touch it. **Three**, walk the retrieval diagnostic in order: is it indexed, is it retrieved, does rephrasing find it, does raising `k` find it, is the chunk itself whole. **Four**, only when all retrieval checks pass do you inspect the prompt and model. **Five**, never trust an answer's confident tone as evidence it is right, and always reproduce the exact failing question rather than debugging from a description. **Six**, change one thing at a time, so every fix teaches you its cause.

You can now diagnose any wrong answer and name its true cause in minutes. The next modules give you the tools to act on those diagnoses: better retrieval when the diagnostic points at ranking or vocabulary, and the observability that makes this whole procedure faster by capturing the chunks and prompt for every request automatically. Debugging is the skill; the rest of the course is the toolbox it reaches for.